

**(A0510194) DESIGN AND ANALYSIS OF ALGORITHMS**

**UNIT II**

**UNIT-II:**

Divide and conquer: General method, applications-Binary search, Finding Maximum and minimum, Quick sort, Merge sort, Strassen's matrix multiplication.

**Divide and conquer:**

Given a function to compute on  $n$  inputs, the divide and conquer strategy suggests splitting the inputs into  $k$  distinct subsets,  $1 < k \leq n$  yielding  $k$  sub problems. These sub problems must be solved and then a method must be found to combine sub solutions into a solution of the whole. If the sub problems are still relatively large, then the divide and conquer strategy may be reapplied.

**Control abstraction:** A procedure whose flow of control is clear but whose primary operations are specified by other procedure whose precise meanings are left undefined.

Control abstraction for divide and conquer:

Algorithm DAndC(P)

```
{
    if Small(P) then return S(P);
    else
    {
        Divide P into smaller instances  $P_1, P_2, \dots, P_k, k \geq 1$ 
        Apply DAndC to each of these subproblems;
        Return Combine(DAndC( $P_1$ ), DAndC( $P_2$ ),  $\dots$ , DAndC( $P_k$ ));
    }
}
```

Small(P) is a boolean valued function that determines whether the input size is small enough that the answer can be computed without splitting.

Combine is a function that determines the solution to P using the solutions to the  $k$  sub problems.

The complexity of many divide and conquer algorithms is given by the recurrences of the form

$$T(n) = \begin{cases} T(1) & n = 1 \\ a T\left(\frac{n}{b}\right) + f(n) & n > 1 \end{cases}$$

If the problem is divided into two sub problems of equal size, then the computing time of DAndC is described as

$$T(n) = \begin{cases} g(n) & \text{if } n \text{ is small} \\ 2 T\left(\frac{n}{2}\right) + f(n) & \text{otherwise} \end{cases}$$

Where  $T(n)$  is the time for DAndC on  $n$  inputs,  $g(n)$  is the time to compute the answer directly for small inputs and  $f(n)$  is the time for Divide and Combine.

**Binary Search (Recursive):**

```
Algorithm BinSrch( $a, i, l, x$ )
// Given an array  $a[i : l]$  of elements in nondecreasing
// order,  $1 \leq i \leq l$ , determine whether  $x$  is present, and
// if so, return  $j$  such that  $x = a[j]$ ; else return 0.
{
    if ( $l = i$ ) then // If Small( $P$ )
    {
        if ( $x = a[i]$ ) then return  $i$ ;
        else return 0;
    }
    else
    { // Reduce  $P$  into a smaller subproblem.
         $mid := \lfloor (i + l) / 2 \rfloor$ ;
        if ( $x = a[mid]$ ) then return  $mid$ ;
        else if ( $x < a[mid]$ ) then
            return BinSrch( $a, i, mid - 1, x$ );
        else return BinSrch( $a, mid + 1, l, x$ );
    }
}
```

**Binary Search (Iterative):**

```
Algorithm BinSearch( $a, n, x$ )
// Given an array  $a[1 : n]$  of elements in nondecreasing
// order,  $n \geq 0$ , determine whether  $x$  is present, and
// if so, return  $j$  such that  $x = a[j]$ ; else return 0.
{
     $low := 1$ ;  $high := n$ ;
    while ( $low \leq high$ ) do
    {
         $mid := \lfloor (low + high) / 2 \rfloor$ ;
        if ( $x < a[mid]$ ) then  $high := mid - 1$ ;
        else if ( $x > a[mid]$ ) then  $low := mid + 1$ ;
        else return  $mid$ ;
    }
    return 0;
}
```

Eg: Let us consider the following 14 elements

-15, -6, 0, 7, 9, 23, 54, 82, 101, 112, 125, 131, 142, 151

| x = 151 |      |       | x = -14 |      |           | x = 9 |      |       |
|---------|------|-------|---------|------|-----------|-------|------|-------|
| low     | high | mid   | low     | high | mid       | low   | high | mid   |
| 1       | 14   | 7     | 1       | 14   | 7         | 1     | 14   | 7     |
| 8       | 14   | 11    | 1       | 6    | 3         | 1     | 6    | 3     |
| 12      | 14   | 13    | 1       | 2    | 1         | 4     | 6    | 5     |
| 14      | 14   | 14    | 2       | 2    | 2         |       |      | Found |
|         |      | Found | 2       | 1    | Not found |       |      |       |

Time complexity of successful search = best  $O(1)$ ,  
average  $O(\log n)$ ,  
Worst  $O(\log n)$ .

Time complexity of unsuccessful search =  $O(\log n)$ .

### **Merge sort:**

Given a sequence of  $n$  elements  $a[1], \dots, a[n]$ . The idea is to split in to two sets  $a[1], \dots, a[n/2]$  and  $a[(n/2)+1], \dots, a[n]$ . Each set is individually sorted, and the resulting sorted sequences are merged to produce a single sorted sequence of  $n$  elements.

```

Algorithm MergeSort(low, high)
// a[low : high] is a global array to be sorted.
// Small(P) is true if there is only one element
// to sort. In this case the list is already sorted.
{
    if (low < high) then // If there are more than one element
    {
        // Divide P into subproblems.
        // Find where to split the set.
        mid :=  $\lfloor (low + high) / 2 \rfloor$ ;
        // Solve the subproblems.
        MergeSort(low, mid);
        MergeSort(mid + 1, high);
        // Combine the solutions.
        Merge(low, mid, high);
    }
}
    
```

```

Algorithm Merge(low, mid, high)
// a[low : high] is a global array containing two sorted
// subsets in a[low : mid] and in a[mid + 1 : high]. The goal
// is to merge these two sets into a single set residing
// in a[low : high]. b[ ] is an auxiliary global array.
{
    h := low; i := low; j := mid + 1;
    while ((h ≤ mid) and (j ≤ high)) do
    {
        if (a[h] ≤ a[j]) then
        {
            b[i] := a[h]; h := h + 1;
        }
        else
        {
            b[i] := a[j]; j := j + 1;
        }
        i := i + 1;
    }
    if (h > mid) then
        for k := j to high do
        {
            b[i] := a[k]; i := i + 1;
        }
    else
        for k := h to mid do
        {
            b[i] := a[k]; i := i + 1;
        }
    for k := low to high do a[k] := b[k];
}

```

Eg: *a*[1:10] = (310, 285, 179, 652, 351, 423, 861, 254, 450, 520)

(310 | 285 | 179 | 652, 351 | 423, 861, 254, 450, 520)

Elements *a*[1] and *a*[2] are merged to yield

(285, 310 | 179 | 652, 351 | 423, 861, 254, 450, 520)

Then  $a[3]$  is merged with  $a[1:2]$

(179,285,310 | 652,351 | 423,861,254, 450,520)

Next, elements  $a[4]$  and  $a[5]$  are merged

(179,285,310 | 351,652 | 423,861,254, 450,520)

and then  $a[1:3]$  and  $a[4 :5]$  are merged

(179,285,310,351,652 | 423,861,254, 450,520)

(179,285,310,351,652 | 423 | 861 | 254 | 450,520)

Elements  $a[6]$  and  $a[7]$  are merged. Then  $a[8]$  is merged with  $a[6:7]$ :

(179,285,310,351,652 | 254, 423,861 | 450,520)

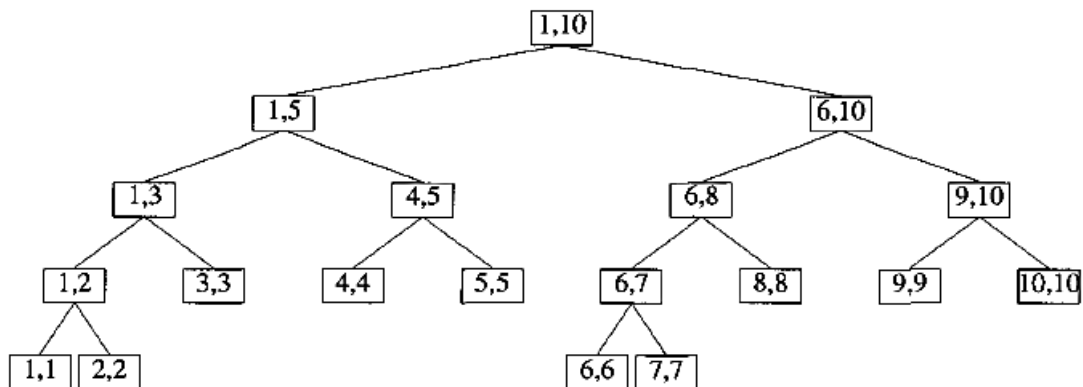
Next  $a[9]$  and  $a[10]$  are merged, and then  $a[6:8]$  and  $a[9:10]$  are merged

(179,285,310,351,652 | 254, 423,450,520,861)

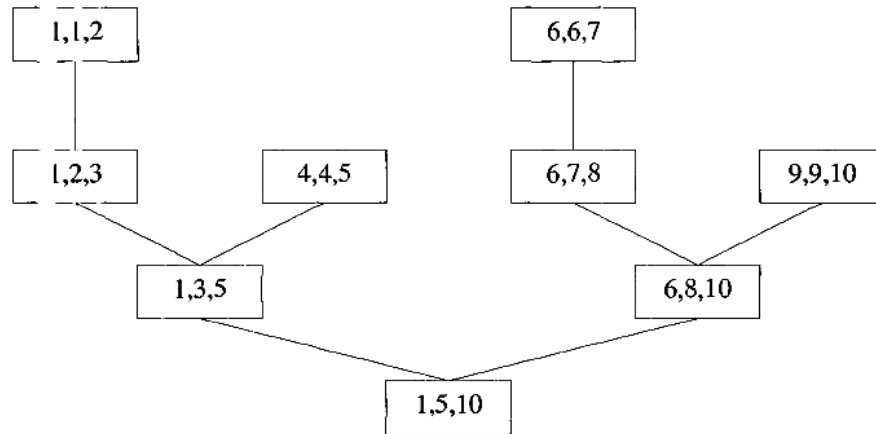
At this point there are two sorted sub arrays and the final merge produces the fully sorted result

(179, 254, 285,310, 351, 423, 450, 520, 652, 861).

Tree of calls of MergeSort(1,10) are shown below:



Trees of calls of Merge are shown below:



If the time for the merging operation is proportional to 'n', then the computing time for merge sort is described by the recurrence relation

$$T(n) = \begin{cases} a & n = 1, \text{ } a \text{ is constant} \\ 2 T\left(\frac{n}{2}\right) + cn & n > 1, \text{ } c \text{ is constant} \end{cases}$$

$$\begin{aligned} T(n) &= 2 T\left(\frac{n}{2}\right) + cn \\ &= 2 \left[ 2 T\left(\frac{n}{4}\right) + \frac{cn}{2} \right] + cn \\ &= 4 T\left(\frac{n}{4}\right) + 2cn \\ &= 4 \left[ 2 T\left(\frac{n}{8}\right) + \frac{cn}{4} \right] + 2cn \\ &= 8 T\left(\frac{n}{8}\right) + 3cn \end{aligned}$$

When n is power of 2,  $n = 2^k$

$$\begin{aligned} T(n) &= 2^k T(1) + k c n \\ &= a n + c n \log n \end{aligned}$$

$\therefore$  The time complexity  $T(n) = O(n \log n)$ .

Disadvantage of Merge sort:

Merge sort uses  $2n$  locations. The additional  $n$  locations are needed because we could not merge two sorted sets in place. On each call of Merge the values from array  $b[]$  are copied back to array  $a[]$ .

To avoid this advantage, we use QUICKSORT.

**Quicksort:**

In merge sort, the array  $a[1:n]$  is divided at its mid-point in to sub arrays which were independently sorted and later merged.

But in Quicksort, the division in to two sub arrays is made so that the sorted sub arrays need not to be merged later. This is accomplished by rearranging the elements in  $a[1:n]$  such that  $a[i] \leq a[j]$  for all  $i$  between 1 and  $m$  and for all  $j$  between  $m+1$  and  $n$  for some  $m$ ,  $1 \leq m \leq n$ .

Thus the elements in  $a[1:m]$  and  $a[m+1:n]$  can be independently sorted. No merge is needed.

The rearrangement of the elements is accomplished by picking some element of  $a$ , say  $t = a[s]$ , and then re ordering the other elements so that all the elements appearing before  $t$  in  $a[1:n]$  are less than or equal to  $t$  and all elements appearing after  $t$  are greater than or equal to  $t$ . This rearranging is called partitioning.

Eg:

| (1) | (2) | (3) | (4) | (5) | (6) | (7) | (8) | (9) | (10)      | $i$ | $p$ |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----------|-----|-----|
| 65  | 70  | 75  | 80  | 85  | 60  | 55  | 50  | 45  | $+\infty$ | 2   | 9   |
| 65  | 45  | 75  | 80  | 85  | 60  | 55  | 50  | 70  | $+\infty$ | 3   | 8   |
| 65  | 45  | 50  | 80  | 85  | 60  | 55  | 75  | 70  | $+\infty$ | 4   | 7   |
| 65  | 45  | 50  | 55  | 85  | 60  | 80  | 75  | 70  | $+\infty$ | 5   | 6   |
| 65  | 45  | 50  | 55  | 60  | 85  | 80  | 75  | 70  | $+\infty$ | 6   | 5   |
| 60  | 45  | 50  | 55  | 65  | 85  | 80  | 75  | 70  | $+\infty$ |     |     |



**Algorithm** Partition( $a, m, p$ )  
 // Within  $a[m], a[m + 1], \dots, a[p - 1]$  the elements are  
 // rearranged in such a manner that if initially  $t = a[m]$ ,  
 // then after completion  $a[q] = t$  for some  $q$  between  $m$   
 // and  $p - 1$ ,  $a[k] \leq t$  for  $m \leq k < q$ , and  $a[k] \geq t$   
 // for  $q < k < p$ .  $q$  is returned. Set  $a[p] = \infty$ .  
 {  
      $v := a[m]$ ;  $i := m$ ;  $j := p$ ;  
     **repeat**  
     {  
         **repeat**  
          $i := i + 1$ ;  
         **until** ( $a[i] \geq v$ );  
  
         **repeat**  
          $j := j - 1$ ;  
         **until** ( $a[j] \leq v$ );  
  
         **if** ( $i < j$ ) **then** Interchange( $a, i, j$ );  
     } **until** ( $i \geq j$ );  
      $a[m] := a[j]$ ;  $a[j] := v$ ; **return**  $j$ ;  
 }

**Algorithm** Interchange( $a, i, j$ )  
 // Exchange  $a[i]$  with  $a[j]$ .  
 {  
      $p := a[i]$ ;  
      $a[i] := a[j]$ ;  $a[j] := p$ ;  
 }

**Algorithm** QuickSort( $p, q$ )  
 // Sorts the elements  $a[p], \dots, a[q]$  which reside in the global  
 // array  $a[1 : n]$  into ascending order;  $a[n + 1]$  is considered to  
 // be defined and must be  $\geq$  all the elements in  $a[1 : n]$ .  
 {  
     **if** ( $p < q$ ) **then** // If there are more than one element  
     {  
         // divide  $P$  into two subproblems.  
          $j := \text{Partition}(a, p, q + 1)$ ;  
         //  $j$  is the position of the partitioning element.  
         // Solve the subproblems.  
         QuickSort( $p, j - 1$ );  
         QuickSort( $j + 1, q$ );  
         // There is no need for combining solutions.  
     }  
 }

Analysis of Quicksort:

Assume that the partitioning element  $v$  has an equal probability of being the  $i$ th smallest element,  $1 \leq i \leq p-m$  in  $a[m:p-1]$ .

The average time complexity of Quicksort  $C_A(n)$  is described as

$$C_A(n) = n + 1 + \frac{1}{n} \sum_{1 \leq k \leq n} [C_A(k-1) + C_A(n-k)] \quad \dots \quad (1)$$

$n+1$  is the number of element comparisons required by partition on its first call.

$$C_A(0) = C_A(1) = 0$$

Multiplying both sides of eq. (1) by  $n$ ,

$$n C_A(n) = n(n+1) + 2[C_A(0) + C_A(1) + \dots + C_A(n-1)] \quad \dots \quad (2)$$

$n$  by ' $n-1$ ' in eq.(2)

$$(n-1) C_A(n-1) = (n-1)n + 2[C_A(0) + C_A(1) + \dots + C_A(n-2)] \quad \dots \quad (3)$$

Subtracting eq. (3) from eq. (2)

$$n C_A(n) - (n-1) C_A(n-1) = 2n + 2C_A(n-1)$$

$$n C_A(n) = 2n + 2C_A(n-1) + (n-1) C_A(n-1)$$

$$n C_A(n) = 2n + (n+1) C_A(n-1)$$

Dividing both sides by  $n(n+1)$

$$\frac{n C_A(n)}{n(n+1)} = \frac{2n}{n(n+1)} + \frac{(n+1) C_A(n-1)}{n(n+1)}$$

$$\Rightarrow \frac{C_A(n)}{(n+1)} = \frac{C_A(n-1)}{n} + \frac{2}{n+1}$$

$$\Rightarrow \frac{C_A(n)}{(n+1)} = \frac{C_A(n-2)}{n-1} + \frac{2}{n} + \frac{2}{n+1}$$

$$\Rightarrow \frac{C_A(n)}{(n+1)} = \frac{C_A(n-3)}{n-2} + \frac{2}{n-1} + \frac{2}{n} + \frac{2}{n+1}$$

⋮

$$\Rightarrow \frac{C_A(n)}{(n+1)} = \frac{C_A(1)}{2} + \frac{2}{3} + \frac{2}{4} + \dots + \frac{2}{n+1}$$

$$\Rightarrow \frac{C_A(n)}{(n+1)} = \frac{C_A(1)}{2} + 2 \sum_{3 \leq k \leq n+1} \frac{1}{k}$$

$$\Rightarrow \frac{C_A(n)}{(n+1)} = 2 \sum_{3 \leq k \leq n+1} \frac{1}{k}$$

Since  $C_A(1) = 0$

$$\sum_{3 \leq k \leq n+1} \frac{1}{k} \leq \int_2^{n+1} \frac{1}{x} dx = \log(n+1) - \log 2$$

$$\frac{C_A(n)}{(n+1)} \leq 2 [\log(n+1) - \log 2]$$

$$C_A(n) \leq 2(n+1)[\log(n+1) - \log 2]$$

$$C_A(n) = O(n \log n).$$

### **Strassen's matrix multiplication:**

Let A and B be two  $n \times n$  matrices. The product matrix  $C = AB$  is also an  $n \times n$  matrix whose  $(i,j)^{\text{th}}$  element is formed by taking the elements in the  $i^{\text{th}}$  row of A and  $j^{\text{th}}$  column of B and multiplying them to get  $C(i,j)$ .

$$C(i,j) = \sum_{1 \leq k \leq n} A(i,k) * B(k,j)$$

To compute  $C(i,j)$  using this formula we need  $n$  multiplications. Since the matrix C has  $n^2$  elements, the time for the resulting matrix multiplication is  $O(n^3)$ .

The divide and conquer strategy suggests another way to compute the product of two  $n \times n$  matrices. For simplicity, Assume  $n = 2^k$ . In case  $n$  is not power of 2, then enough rows and columns of 0's can be added to both A and B, so that the resulting dimensions are a power of 2.

Imagine that A and B are each partitioned in to 4 square sub matrices, each matrix having dimensions  $\frac{n}{2} \times \frac{n}{2}$ . Then the product AB is computed as

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

Then

$$C_{11} = A_{11} B_{11} + A_{12} B_{21}$$

$$C_{12} = A_{11} B_{12} + A_{12} B_{22}$$

$$C_{21} = A_{21} B_{11} + A_{22} B_{21}$$

$$C_{22} = A_{21} B_{12} + A_{22} B_{22}$$

To compute AB, we need to perform 8 multiplications of  $\frac{n}{2} \times \frac{n}{2}$  matrices and 4 additions of  $\frac{n}{2} \times \frac{n}{2}$  matrices. Since two  $\frac{n}{2} \times \frac{n}{2}$  matrices can be added in time  $cn^2$  for some constant c, the overall computing time  $T(n)$  is given by the recurrence relation

$$T(n) = \begin{cases} b & n \leq 2 \\ 8T\left(\frac{n}{2}\right) + cn^2 & n > 2 \end{cases}$$

$$\begin{aligned} T(n) &= 8T\left(\frac{n}{2}\right) + cn^2 \\ &= 8\left[8T\left(\frac{n}{4}\right) + \frac{cn^2}{4}\right] + cn^2 \\ &= 64T\left(\frac{n}{4}\right) + 3cn^2 \\ &= (2^k)^3 + \frac{n}{2^k} + (2^k - 1)cn^2 \\ &= bn^3 + (n - 1)cn^2 \\ &= O(n^3) \end{aligned}$$

Hence no improvement over the conventional method has been made. Matrix multiplications are more expensive than matrix additions.

Strassen has discovered a way to compute  $C_{ij}$  using only 7 multiplications and 18 additions or subtractions. His method involves first computing the seven  $\frac{n}{2} \times \frac{n}{2}$  matrices P, Q, R, S, T, U and V. Then  $C_{ij}$ 's are computed using the matrices P, Q, R, S, T, U and V.

$$P = (A_{11} + A_{22}) (B_{11} + B_{22})$$

$$Q = (A_{21} + A_{22}) B_{11}$$

$$R = A_{11} (B_{12} - B_{22})$$

$$S = A_{22} (B_{21} - B_{11})$$

$$T = (A_{11} + A_{12}) B_{22}$$

$$U = (A_{21} - A_{11}) (B_{11} + B_{12})$$

$$T = (A_{12} - A_{22}) (B_{21} + B_{22})$$

Then

$$C_{11} = P + S - T + V$$

$$C_{12} = R + T$$

$$C_{21} = Q + S$$

$$C_{22} = P + R - Q + U$$

The resulting recurrence relation for  $T(n)$  is

$$T(n) = \begin{cases} b & n \leq 2 \\ 7T\left(\frac{n}{2}\right) + an^2 & n > 2 \end{cases}$$

$$\begin{aligned} T(n) &= 7T\left(\frac{n}{2}\right) + an^2 \\ &= 7\left[7T\left(\frac{n}{4}\right) + a\left(\frac{n}{2}\right)^2\right] + an^2 \\ &= 49T\left(\frac{n}{4}\right) + \frac{7}{4}an^2 + an^2 \\ &= an^2\left[1 + \frac{7}{4} + \left(\frac{7}{4}\right)^2 + \dots + \left(\frac{7}{4}\right)^{k-1}\right] + 7^k T(1) \\ &\leq cn^2\left(\frac{7}{4}\right)^{\log n} + b \cdot 7^{\log n} \\ &\leq cn^{\log 4 + \log 7 - \log 4} + b \cdot n^{\log 7} \\ &= O(n^{\log 7}) \\ &= O(n^{2.81}) \end{aligned}$$

**Finding the Maximum and minimum:**

The problem is to find the maximum and minimum items in a set of  $n$  elements.

The straight forward algorithm to accomplish this is shown below:

```
Algorithm StraightMaxMin( $a, n, max, min$ )  
// Set  $max$  to the maximum and  $min$  to the minimum of  $a[1 : n]$ .  
{  
     $max := min := a[1]$ ;  
    for  $i := 2$  to  $n$  do  
    {  
        if ( $a[i] > max$ ) then  $max := a[i]$ ;  
        if ( $a[i] < min$ ) then  $min := a[i]$ ;  
    }  
}
```

The algorithm StraightMaxMin requires  $2(n-1)$  element comparisons in the best, average and worst cases.

An immediate improvement is possible by realizing that the comparison  $a[i] < min$  is necessary only when  $a[i] > max$  is false. Hence we may replace the contents of the for loop by

```
if ( $a[i] > max$ ) then  $max := a[i]$ ;  
else if ( $a[i] < min$ ) then  $min := a[i]$ ;
```

Now the best case occurs when the elements are in increasing order. The number of element comparisons is  $n-1$ . The worst case occurs when the elements are in decreasing order. In this case, the number of element comparisons is  $2(n-1)$ . The average number of element comparisons is less than  $2n - 1$ . On the average,  $a[i]$  will be greater than  $max$  half of the time and so the average number of comparisons is  $3n/2 - 1$ . [  $n-1 + n/2$  ]

A divide and conquer algorithm for this problem would proceed as follows:

Let  $P = (n, a[i], \dots, a[j])$  denote an arbitrary instance of the problem. Here  $n$  is the number of elements in the list  $a[i], \dots, a[j]$  and we are interested in finding the maximum and minimum of this list. Let  $Small(P)$  be true when  $n < 2$ . In this

case, the maximum and minimum are  $a[i]$  if  $n = 1$ . If  $n = 2$ , the problem can be solved by making one comparison.

If the list has more than two elements,  $P$  has to be divided into smaller instances. For example, we might divide  $P$  into the two instances  $P_1 = (n/2, a[1], \dots, a[n/2])$  and  $P_2 = (n - [n/2], a[n/2+1], \dots, a[n])$ . After having divided  $P$  into two smaller sub problems, we can solve them by recursively invoking the same divide-and-conquer algorithm. How can we combine the solutions for  $P_1$  and  $P_2$  to obtain a solution for  $P$ ? If  $\text{MAX}(P)$  and  $\text{MIN}(P)$  are the maximum and minimum of the elements in  $P$ , then  $\text{MAX}(P)$  is the larger of  $\text{MAX}(P_1)$  and  $\text{MAX}(P_2)$ . Also,  $\text{MIN}(P)$  is the smaller of  $\text{MIN}(P_1)$  and  $\text{MIN}(P_2)$ .

```

Algorithm MaxMin( $i, j, max, min$ )
//  $a[1 : n]$  is a global array. Parameters  $i$  and  $j$  are integers,
//  $1 \leq i \leq j \leq n$ . The effect is to set  $max$  and  $min$  to the
// largest and smallest values in  $a[i : j]$ , respectively.
{
    if ( $i = j$ ) then  $max := min := a[i]$ ; // Small( $P$ )
    else if ( $i = j - 1$ ) then // Another case of Small( $P$ )
        {
            if ( $a[i] < a[j]$ ) then
            {
                 $max := a[j]$ ;  $min := a[i]$ ;
            }
            else
            {
                 $max := a[i]$ ;  $min := a[j]$ ;
            }
        }
    else
    { // If  $P$  is not small, divide  $P$  into subproblems.
      // Find where to split the set.
         $mid := \lfloor (i + j) / 2 \rfloor$ ;
      // Solve the subproblems.
        MaxMin( $i, mid, max, min$ );
        MaxMin( $mid + 1, j, max1, min1$ );
      // Combine the solutions.
        if ( $max < max1$ ) then  $max := max1$ ;
        if ( $min > min1$ ) then  $min := min1$ ;
    }
}

```

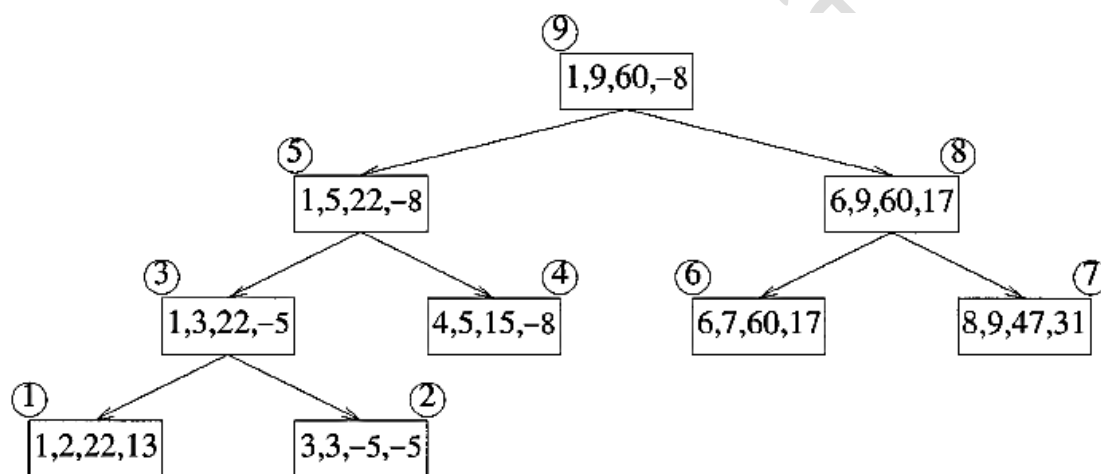
For sets containing more than two elements, the midpoint is determined (just as in binary search) and two new sub problems are generated. When the maxima and minima of these sub problems are determined, the two maxima are compared and the two minima are compared to achieve the solution for the entire set.

The procedure is initially invoked by the statement  $\text{MaxMin}(l,n,x,y)$ .

Suppose we simulate  $\text{MaxMin}$  on the following 9 elements.

a:    [1]   [2]   [3]   [4]   [5]   [6]   [7]   [8]   [9]  
          22   13   -5   -8   15   60   17   31   47

The following figure shows the trees of recursive calls of  $\text{MaxMin}$ .



If  $T(n)$  represents the number of element comparisons needed for  $\text{MaxMin}$ , then the resulting recurrence relation is

$$T(n) = \begin{cases} T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + 2 & n > 2 \\ 1 & n = 2 \\ 0 & n = 1 \end{cases}$$

$$T(n) = 2 T\left(\frac{n}{2}\right) + 2$$

$$T(n) = 2 \left[ 2 T\left(\frac{n}{4}\right) + 2 \right] + 2$$



$$T(n) = 4 T\left(\frac{n}{4}\right) + 4 + 2$$

$\vdots$

$$T(n) = 2^{k-1} T(2) + \sum_{1 \leq i \leq k-1} 2^i$$

$$T(n) = 2^{k-1} + 2^k - 2$$

$$T(n) = \frac{n}{2} + n - 2$$

$$T(n) = \frac{3n}{2} - 2$$

$3n/2 - 2$  is the best, average and worst case number of comparisons when  $n$  is power of 2.